Year: 2026-2027 Sem V-NEP
Course: Agile Software Development & DevOps Lab
Class: T1 Date of Submission: 15/10/26

A Mini Project Report on

Gym Membership Management

A Gym Membership Management System Delivered Through an Agile and DevOps Workflow

Submitted in partial fulfilment of the requirements of the course
Agile Software Development & DevOps Lab
Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	**Roll No.**
Ayush Kushwaha	2401054
Palak Lakhani	2401056
Vedprakash Mishra	2401066

Under the guidance of
Prof. _____
Department of AI & DS
Thadomal Shahani Engineering College
2026–2027

CERTIFICATE

This is to certify that the mini project entitled “Gym Membership Management: A Gym Membership Management System Delivered Through an Agile and DevOps Workflow” is a bona fide work of Ayush Kushwaha (2401054), Palak Lakhani (2401056) and Vedprakash Mishra (2401066), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

_____

Project Guide

_____

Head of Department

Date: _____

Place: _____

ABSTRACT

Gym Membership Management is a web application for maintaining member, subscription plan and staff records. The interface provides separate lists and forms for these records. Each member is linked to an existing plan, and the application stores changes in MySQL. The frontend uses HTML, CSS and JavaScript, while a Spring Boot backend provides the APIs used by the browser.

The project applies version control, containerization, continuous integration, deployment, Agile planning and monitoring to one application. GitHub stores the source code and merged pull requests. Docker Compose runs the application, MySQL, Prometheus, node-exporter and Grafana. Jenkins runs six automated tests and includes image building, deployment and a health check. Jira contains four epics and seven stories. Prometheus collects application and system measurements, which Grafana displays in a dashboard. The report presents the working services, the recorded build results and the remaining delivery checks.

Keywords: Agile, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Jira, Prometheus, Grafana, Spring Boot, MySQL

CONTENTS

Chapter 1 Introduction

Chapter 2 Requirements and Agile Planning — LO 1

Chapter 3 System Design

Chapter 4 Application Implementation

Chapter 5 DevOps Implementation — LO 1 to LO 5

Chapter 6 Testing

Chapter 7 Implementation

Chapter 8 Results and Discussion

Chapter 9 Conclusion and Future Scope — LO 6

Appendix A Running the Project

1. Introduction

1.1 Background

Gym Membership Management is a web-based application for managing the members, subscription plans and staff of a gym. In many small gyms, this information is kept in paper registers or spreadsheets, which makes it hard to know which plan a member holds, how many members are active and who is working at the gym. The application moves this information online into one system backed by a database.

The Agile Software Development & DevOps Lab project uses this application to connect source control, testing, container deployment, task planning and monitoring. The same source files are used by the Docker build and the Jenkins pipeline, while Jira records the work required to deliver and verify them.

1.2 Problem Statement

To develop and deliver a gym records application with persistent storage, version-controlled source code, automated tests, container deployment, Agile task tracking and application monitoring.

1.3 Objectives

1. Provide a gym management application in which members, subscription plans and staff can be stored and viewed.
2. Use Git and GitHub for commits, feature branches, pull requests and merge history.
3. Containerize the Spring Boot application using Docker and run the application, MySQL and the monitoring stack as interconnected services using Docker Compose.
4. Configure Jenkins for Continuous Integration and extend the pipeline for automated Docker image building, deployment and smoke testing.
5. Use Jira for epics, stories, estimates, sprint planning and dependencies.
6. Integrate Prometheus and Grafana to collect and visualise application metrics and server (CPU, memory, disk) metrics.

1.4 Scope

The application scope covers Members, Plans and Staff. Payment collection, attendance, member login and automatic membership expiry are not implemented. The deployment runs locally using Docker Desktop and Windows Jenkins. Kubernetes, cloud hosting, MLflow and Airflow are outside this implementation.

1.5 Mapping of Lab Topics and Learning Outcomes

No.	Lab Topic	Application in Gym Membership Management
1	Version Control using Git and GitHub (LO 2)	Repository, commits, feature branches, two merged pull requests and local merge history
2	Containerization using Docker (LO 4)	Multi-stage Dockerfile (test and production), image, container and port mapping
3	Multi-Service Deployment using Docker Compose (LO 4)	Spring Boot app + MySQL + Prometheus + node-exporter + Grafana, volumes, health check and service networking
4	Continuous Integration using Jenkins (LO 3)	Jenkins pipeline from the Jenkinsfile, automated tests with JUnit report, GitHub SCM polling
5	Automated Deployment using GitHub and Jenkins (LO 3, LO 6)	Docker image build, Docker Compose deployment and /health smoke test
6	Agile Software Development Workflow using Jira (LO 1)	Four epics, seven stories, one planned sprint and two dependency links
7	Continuous Monitoring using Prometheus (LO 5)	Application metrics, /metrics endpoint, PromQL, Grafana dashboard with CPU, memory and disk

Table 1.1: Lab topics, LO numbers and their application in the project

Official lab outcomes and project coverage

LO 1 — Apply Agile methodologies and use Jira for sprint planning, task tracking, and integrating with DevOps workflows.

LO 2 — Use Git effectively for version control, source code management, and collaborative team development.

LO 3 — Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.

LO 4 — Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO 5 — Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

LO 6 — Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

Source: Semester V syllabus, Agile Software Development and DevOps Lab, course 2015119, PDF pages 36–37. Topic numbering is separate from the syllabus experiment numbering. LO 4 and LO 5 are covered through their Docker/Compose and monitoring components respectively.

2. Requirements and Agile Planning — LO 1

2.1 User Roles

Role	Main Functionality
Gym Administrator	Views and manages the member directory, the subscription plans and the staff records through the web interface
Member (data entity)	Holds a unique member ID, a name, a subscription plan and a membership status (for example Active)
Staff (data entity)	Gym employees stored with their role, for example trainer or receptionist

Table 2.1: User roles and entities

2.2 Functional Requirements

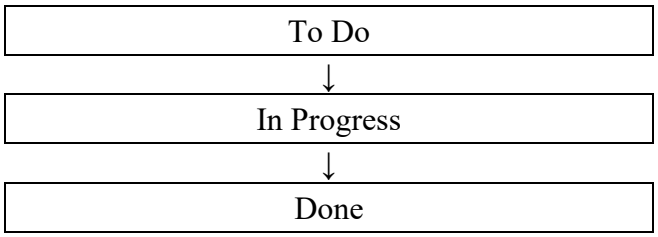
- The system stores gym members with member ID, full name, subscription plan and membership status.
- The system stores subscription plans (for example Monthly, Quarterly and Yearly) with their price and duration.
- The system stores staff IDs, names, specializations and shift timings.
- Each member is linked to a subscription plan through a foreign key.
- The administrator can add, view, update, delete and search records through the web interface.
- The application exposes REST APIs for members, plans and staff, used by the frontend.
- The application exposes /health for readiness checks and /metrics for monitoring.

2.3 Non-Functional / DevOps Requirements

- The application should run in a consistent environment through Docker.
- The Spring Boot application and MySQL should communicate as interconnected services through Docker Compose, with persistent database storage.
- Jenkins should automate source retrieval, building and testing, with deployment and a smoke test added to the extended pipeline.
- Jira should provide an Agile/Scrum workflow for development tasks.
- Prometheus should collect application metrics exposed by the Spring Boot application and server metrics from node-exporter, and Grafana should visualise them.
- Database credentials must be supplied through environment variables and must not be stored in the repository.

2.4 Scrum-Style Workflow

The Jira Scrum project is named Gym Membership Management, with the key GYM. It contains four epics and seven stories. GYM Sprint 1 is prepared but has not been started or given dates. The backlog view displays three open stories with 13 story points; four other stories are marked Done in the project list. The task statuses provide the following progression:



Jira tracks the delivery work separately from the application data. Changing a member record does not update a Jira story. Task owners update their status and attach evidence after testing or reviewing a change.

2.5 Agile-to-DevOps Traceability

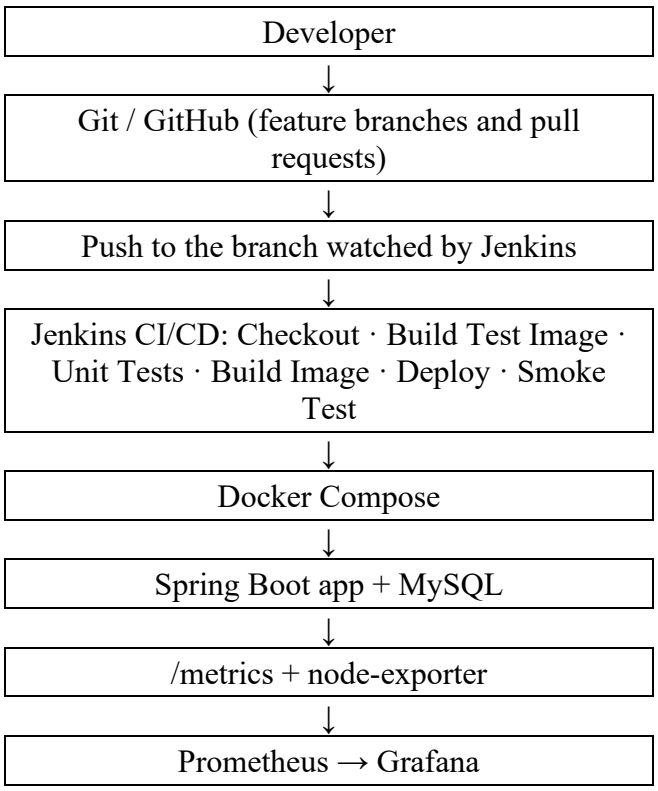
Stage	Tool / Practice	Outcome
Planning	Jira	Epics and stories defined, estimated in story points and planned into a sprint
Development	Git / GitHub	Source code versioned on feature branches and merged through pull requests
Verification	Jenkins	Six automated tests in Jenkins; job watches feature/jenkins
Deployment	Docker / Docker Compose	Application deployed in containers and checked through /health
Monitoring	Prometheus / Grafana	Application and server metrics collected and visualised
Completion	Jira	Owners manually update Jira after verification

Table 2.2: Agile and DevOps workflow traceability

3. System Design

3.1 Architecture

The browser calls the Spring Boot API using relative URLs. The backend stores and retrieves data from MySQL. Docker Compose provides the service network and persistent volumes. Jenkins retrieves source code from GitHub, runs tests and deploys the app. Prometheus collects measurements from the app and node-exporter; Grafana displays those measurements. Jira records the planned work and its status.



Jira → planning, ownership and manual progress tracking

3.2 Technology Stack

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Provides the web interface for the gym administrator
Backend	Java 21, Spring Boot, Spring Data JPA	Implements the application logic and REST APIs for members, plans and staff
Database	MySQL 8	Stores members, subscription plans and staff

Layer	Technology	Purpose
Testing	JUnit with an in-memory H2 test database	Automated tests that run without a real database
Version Control	Git and GitHub	Tracks changes, branches, commits, pull requests and remote source code
Containerization	Docker (multi-stage)	Packages the application and its dependencies
Orchestration	Docker Compose	Runs the app, MySQL, Prometheus, node-exporter and Grafana as interconnected services
CI/CD	Jenkins (Jenkinsfile pipeline)	Automates testing, Docker build, deployment and smoke test
Agile Management	Jira	Plans and tracks development tasks
Monitoring	Spring Boot Actuator, Micrometer, Prometheus, node-exporter, Grafana	Collects application metrics from /metrics and server CPU, memory and disk metrics

Table 3.1: Technology stack

3.3 Service Design

Service	Responsibility
app (Spring Boot)	Runs the Gym Membership Management application on port 8080 and exposes the application pages, REST APIs, /health and /metrics
db (MySQL 8)	Stores the application data in a named Docker volume so that data persists across restarts; a health check gates the app service
prometheus	Scrapes app:8080/metrics, node-exporter:9100 and itself every 15 seconds
node-exporter	Exposes Docker Linux VM CPU, memory and disk measurements
grafana	Shows the provisioned dashboard “GymMS Overview”

Table 3.2: Service design

3.4 Monitoring Design

Spring Boot Actuator and the Micrometer Prometheus registry were added to the application to expose HTTP request metrics, request-duration histograms and JVM metrics through the /metrics endpoint. Four custom gym gauges were added on top of these to report counts taken from the database. Prometheus and node-exporter were added to the Docker Compose environment, and Prometheus reaches the application through the Compose network.

Prometheus collects data every 15 seconds from app:8080/metrics, node-exporter:9100 and itself. The Grafana dashboard combines app status, API request rate, p95 latency, database record counts and resource usage. The resource panels describe Docker's Linux virtual machine, not the whole Windows host.

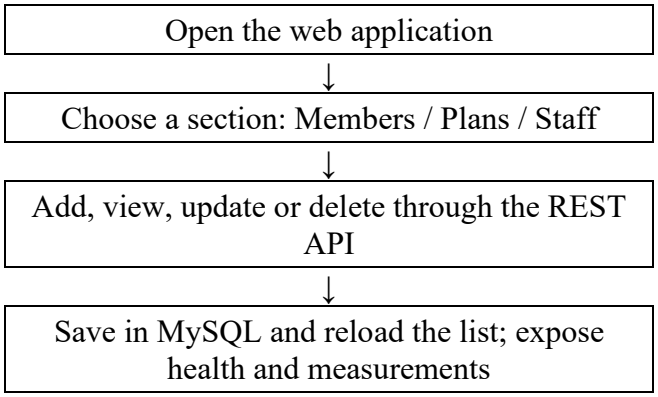
The custom gauges are gym_members_count, gym_members_active, gym_plans_count and gym_staff_count. They read counts from existing tables. Updating a name does not change a record count; changing Active to Expired reduces the active-member count.

4. Application Implementation

4.1 Application Features

The application has a Member Directory, a Subscription Plans list and a Trainers & Staff list. Each list is loaded through the backend API. Forms call the same API to save changes. Members reference a real subscription plan. Sample plans and records are seeded at startup; missing predefined sample records can be recreated on a later restart.

4.2 Workflow of the Application



4.3 Project Repository and Application Structure

The project uses the GitHub repository Ayushk1248/Gym-Membership-Management. The root pom.xml and src/ directory are the build inputs. The separate gym-management/ directory is retained but is not copied into the Docker build. The frontend files are copied into Spring Boot static resources when the image is built.

```
Gym-Membership-Management/
|-- src/                 Spring Boot application (canonical backend)
|-- gym-management/      retained secondary backend copy
|-- monitoring/          prometheus configuration, grafana provisioning +
dashboard
|-- docs/                phase-wise run and verification notes
|-- Dockerfile           multi-stage image (test + production)
|-- docker-compose.yml   app, db, prometheus, node-exporter, grafana
|-- Jenkinsfile          CI/CD pipeline
|-- .env.example         template for environment variables
`-- pom.xml
```

Figure 4.1: Project repository structure

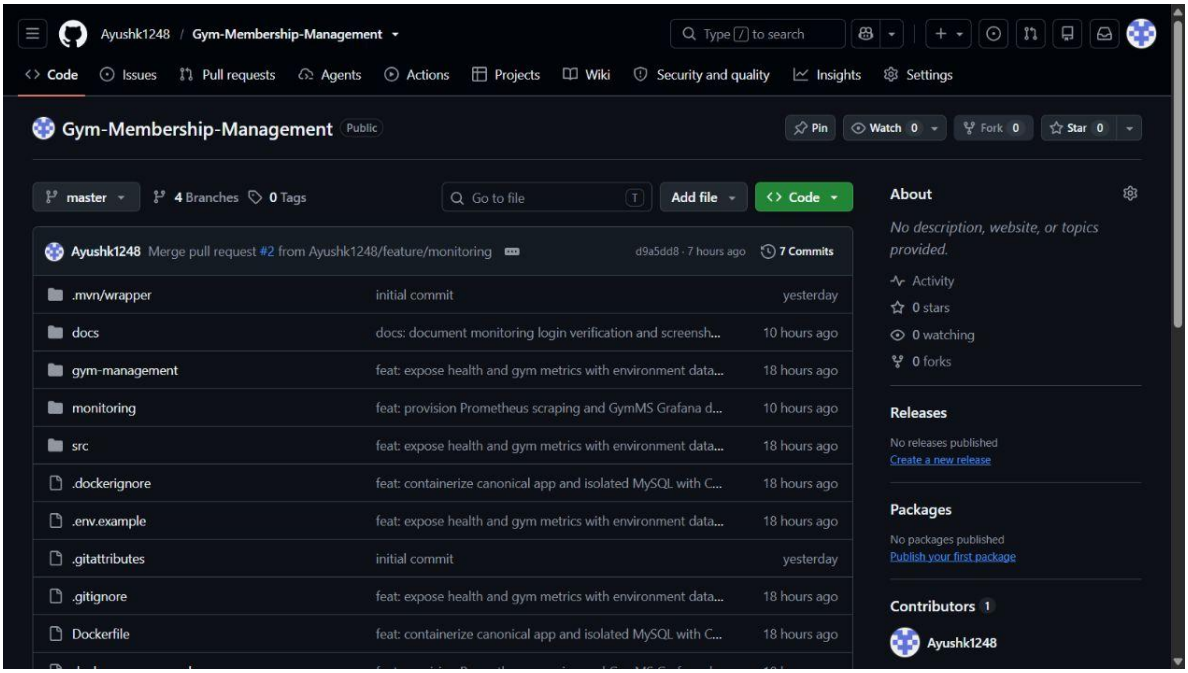


Figure 4.2: GitHub repository on the remote master branch

5. DevOps Implementation

5.1 Version Control Using Git and GitHub — LO 2

Git tracks the Spring Boot backend, frontend, tests and deployment configuration. The remote repository is https://github.com/Ayushk1248/Gym-Membership-Management. The repository name was corrected without changing the local application ports or Docker project name.

The feature/docker branch was merged through Pull Request #1, and feature/monitoring was merged through Pull Request #2. Tests, CRUD validation and the Jenkinsfile are on feature/jenkins. The local master has a merge commit for that branch, while the remote master still points to the Phase 2 merge. Only two pull requests are present on GitHub.

The named test pipelineRevertDemoRejectsMembersWithoutAnExistingPlan is available for a later failed-test and git revert demonstration. The recorded history does not yet contain that demonstration. Git revert would create a new commit that reverses a code change; it would not restore records deleted from MySQL.

LO 2: Use Git effectively for version control, source code management, and collaborative team development.

```
*   5be01a8 Merge feature/jenkins: reliable CRUD and seeded demo data
|\
| * d59779c Seed persistent sample gym records
| * 767e20f Complete CRUD reliability and Jenkins pipeline
|/
*   d9a5dd8 Merge pull request #2 from Ayushk1248/feature/monitoring
|\
| * 2119d7c docs: document monitoring login verification and screenshot checklist
| * 895ddfb feat: provision Prometheus scraping and GymMS Grafana dashboard
|/
*   b673906 Merge pull request #1 from Ayushk1248/feature/docker
|\
| * 845d585 feat: containerize canonical app and isolated MySQL with Compose
| * 5ac9d8f feat: expose health and gym metrics with environment database
credentials
|/
* 8f82cff initial commit
```

Figure 5.1: Actual Git history from the local repository

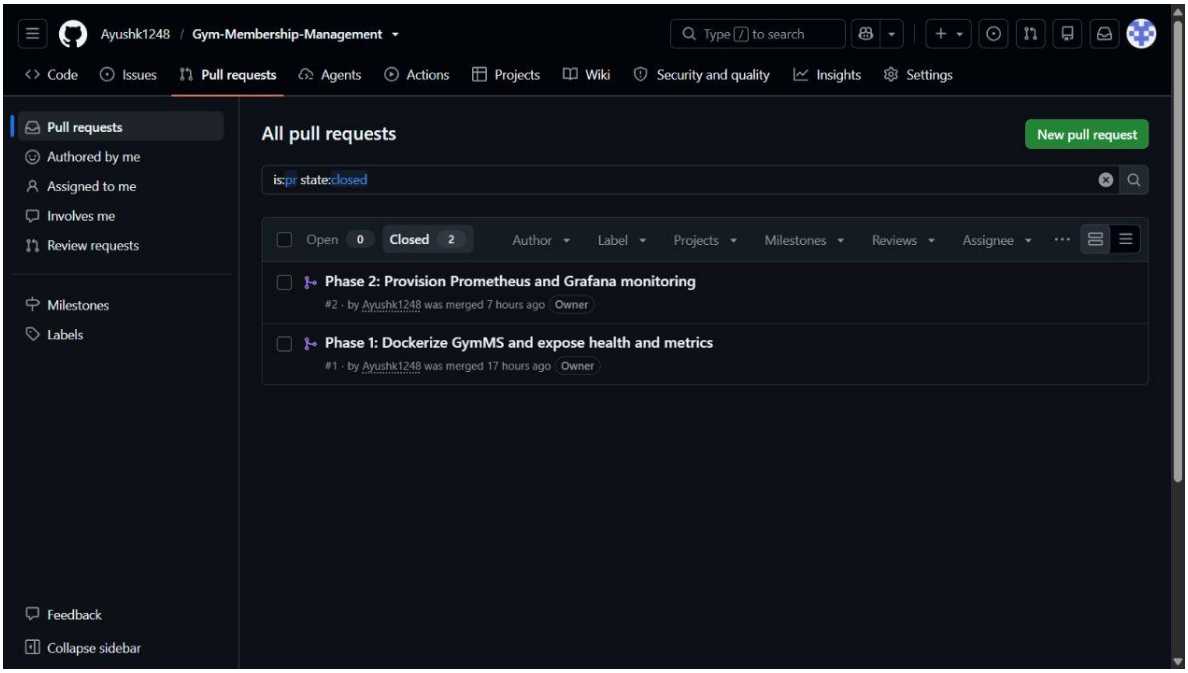


Figure 5.2: The two merged GitHub pull requests

5.2 Containerization Using Docker — LO 4

Docker packages an application together with its dependencies, libraries, runtime and configuration into a consistent environment. A Docker image acts as the template, while a container is a running instance created from that image.

For this project, a multi-stage Dockerfile was created using a Java 21 base image. The build stage compiles the Spring Boot application with Maven. The test stage is used by Jenkins to run the unit tests. The production stage copies only the built application, runs as a non-root user, exposes port 8080 and defines a HEALTHCHECK on the /health endpoint.

The image was built using:

```
docker build --target production -t gymms-app:local .
```

The running app container is gymms-app-1. Its port 8080 is published on 127.0.0.1:8080. The production image uses user 10001:10001 and a health check that requests /health. The Java 21 JRE image contains the packaged application rather than the full source tree.

LO 4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools. This project demonstrates the Docker and Compose part of this outcome.

```
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/* \
    && groupadd --gid 10001 gym && useradd --uid 10001 --gid gym --no-create-home
gym
WORKDIR /app
COPY --from=build --chown=gym:gym /workspace/target/gym-management-0.0.1-
SNAPSHOT.jar app.jar
USER 10001:10001
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=5s --start-period=90s --retries=5 \
    CMD curl --fail --silent http://localhost:8080/health || exit 1
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Figure 5.3: Production Dockerfile excerpt with non-root user and health check

5.3 Multi-Service Deployment Using Docker Compose — LO 4

Docker Compose extends Docker to multi-service applications by allowing related services to be declared in a single YAML configuration file. Each service runs in its own container while communicating with the other services.

The application requires both an application server and a database, and the monitoring stack runs alongside them, so docker-compose.yml was created with five services: the app runs the Spring Boot application, db uses MySQL, and prometheus, node-exporter and grafana provide monitoring. The application communicates with MySQL using the Compose service name and environment variables for the database URL and credentials. The credentials are read from a git-ignored .env file, and a template named .env.example is committed instead.

The mysql_data volume stores database files, and prometheus_data and grafana_data store monitoring data. The application waits for the database health check. Credentials are supplied through environment variables; Jenkins obtains the Compose environment file from its gymms-compose-env credential. The database port is not published to Windows.

LO 4: Docker Compose coordinates the containers, service network, persistent volumes and health checks. Kubernetes and cloud provisioning are not part of this local deployment.

```
SERVICE          IMAGE                          STATE     <no value>
app              gymms-app:local                running   healthy
db               mysql:8.0                      running   healthy
grafana          grafana/grafana:11.6.0         running   healthy
node-exporter    prom/node-exporter:v1.9.1      running
prometheus       prom/prometheus:v3.2.1         running   healthy
```

Figure 5.4: Actual Docker Compose service status

```
gymms_grafana_data
gymms_mysql_data
gymms_prometheus_data
```

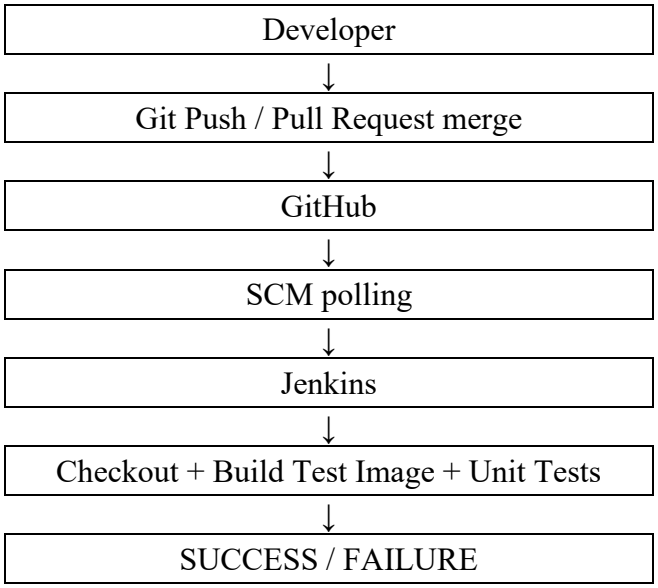
Figure 5.5: Named volumes retained by the gymms stack

5.4 Continuous Integration Using Jenkins — LO 3

Continuous Integration is a development practice in which developers frequently integrate changes into a shared repository and automated processes build and test those changes. Jenkins is an automation server used for CI/CD activities such as building applications, executing tests and performing deployment operations.

The existing Windows Jenkins installation runs on port 8081. Its GymMS-CI-CD job loads Jenkinsfile from https://github.com/Ayushk1248/Gym-Membership-Management.git. The configured branch is */feature/jenkins. Jenkins is not containerized; its bat steps call Docker Desktop.

SCM polling is configured as */2 * * * *, which checks for changes every two minutes while Jenkins is running. A push only to master does not trigger this job while it watches feature/jenkins. Build #1 was started manually and build #2 was started by an SCM change.



LO 3: Build and manage automated CI/CD pipelines for application development, testing, packaging, and deployment.

```
Repository: https://github.com/Ayushk1248/
            Gym-Membership-Management.git
Branch:     */feature/jenkins
Poll SCM:   */2 * * * *
Script:     Jenkinsfile
Host:       Windows Jenkins, localhost:8081
```

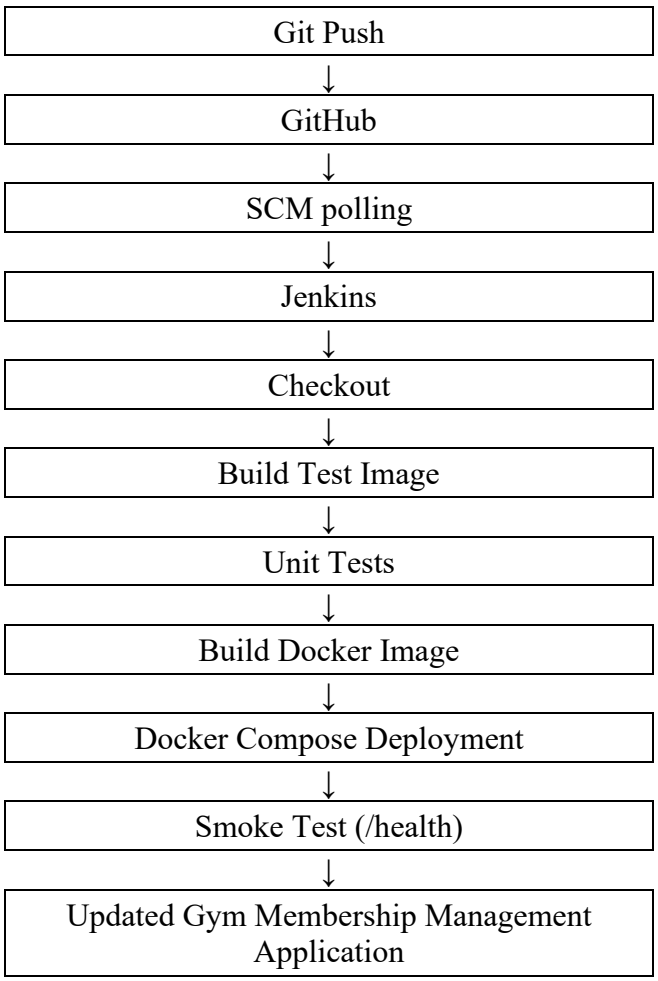
Figure 5.6: Verified Jenkins job configuration

```
Jenkins build #1 — JUnit report
Passed: 6
Failed: 0
Skipped: 0
Overall build result: SUCCESS
```

Figure 5.7: Test summary retrieved from Jenkins build #1

5.5 Automated Deployment Using GitHub and Jenkins — LO 3 and LO 6

This extends Continuous Integration toward Continuous Deployment. The Jenkins pipeline was extended to perform Docker image building, deployment and a smoke test. The resulting workflow is:



After unit tests pass, Jenkins builds gymms-app:local and runs Docker Compose with project name gymms. This updates the same stack used for manual runs and keeps its database volume. The final stage requests /health on port 8080. A failed unit test prevents later stages from running. A failure after deployment does not automatically restore the previous image.

The Jenkins API records the following two completed builds. These are actual results rather than a proposed demonstration sequence.

Build	Started by	Change	Result
#1	User (Build Now)	Pipeline run on feature/jenkins	SUCCESS; 6 tests passed
#2	SCM change	Subsequent feature/jenkins change	FAILURE at Smoke Test; Windows timeout error

Table 5.1: Jenkins builds of GymMS-CI-CD

Build #1 completed successfully with six tests passed. Build #2 reached Smoke Test and failed; its console records "Input redirection is not supported" from the Windows timeout command. This failure occurred after deployment and is not a deliberate unit-test failure. The health endpoint was subsequently reachable. The smoke-test wait needs correction and verification in another build.

LO 3 and LO 6: The pipeline connects source retrieval, testing, packaging, deployment and monitoring. The outstanding smoke-test correction limits the current end-to-end verification.

```
[Pipeline] stage
[Pipeline] { (Smoke Test)
ERROR: Input redirection is not supported,
       exiting the process immediately.
...
ERROR: script returned exit code 1
Finished: FAILURE
```

Figure 5.8: Console excerpt from the actual failed build #2

5.6 Agile Software Development Workflow Using Jira — LO 1

Jira provides a record of the project work, task ownership and dependencies. It is separate from the gym records stored in MySQL. The screenshots in this section are from the live GYM project.

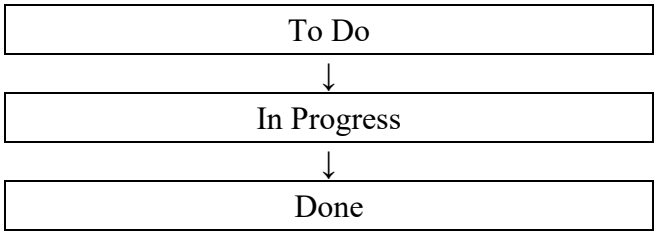
The four epics are Membership Application, Containerization, CI/CD Pipeline and Monitoring. Seven stories describe the application, tests, pipeline, containerization and monitoring work. The current sprint view shows GYM-5, GYM-8 and GYM-9 as To Do, with five, three and five points respectively.

Epic	Stories	Status	Sprint
GYM-1 Membership Application	GYM-5 Manage members, plans and staff with seeded sample data	To Do	GYM Sprint 1
GYM-2 Containerization	GYM-6 Build multi-stage Docker image; GYM-7 Run app, MySQL, Prometheus, node-exporter and Grafana with Docker Compose	Done	GYM Sprint 1

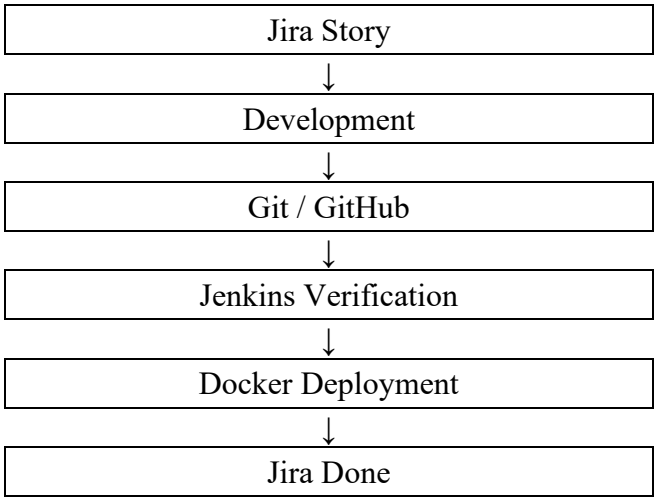
Epic	Stories	Status	Sprint
GYM-3 CI/CD Pipeline	GYM-8 Write automated unit tests; GYM-9 Create Jenkins pipeline GymMS-CI-CD	To Do	GYM Sprint 1
GYM-4 Monitoring	GYM-10 Expose /health and /metrics with gym metrics; GYM-11 Configure Prometheus scraping and provisioned Grafana dashboard	Done	GYM Sprint 1

Table 5.2: Product backlog (epics and stories)

Two dependency links were verified in Jira: GYM-9 is blocked by GYM-8, and GYM-11 is blocked by GYM-10. These links describe the order in which related work should be completed. The intended status progression is:



The intended task flow is shown below. Jira status changes are manual; the Jenkinsfile contains no step that updates a Jira issue.



Issue	Link	Depends on
GYM-9	is blocked by	GYM-8
GYM-11	is blocked by	GYM-10

Table 5.3: Issue dependencies

LO 1: Apply Agile methodologies and use Jira for sprint planning, task tracking, and integrating with DevOps workflows.

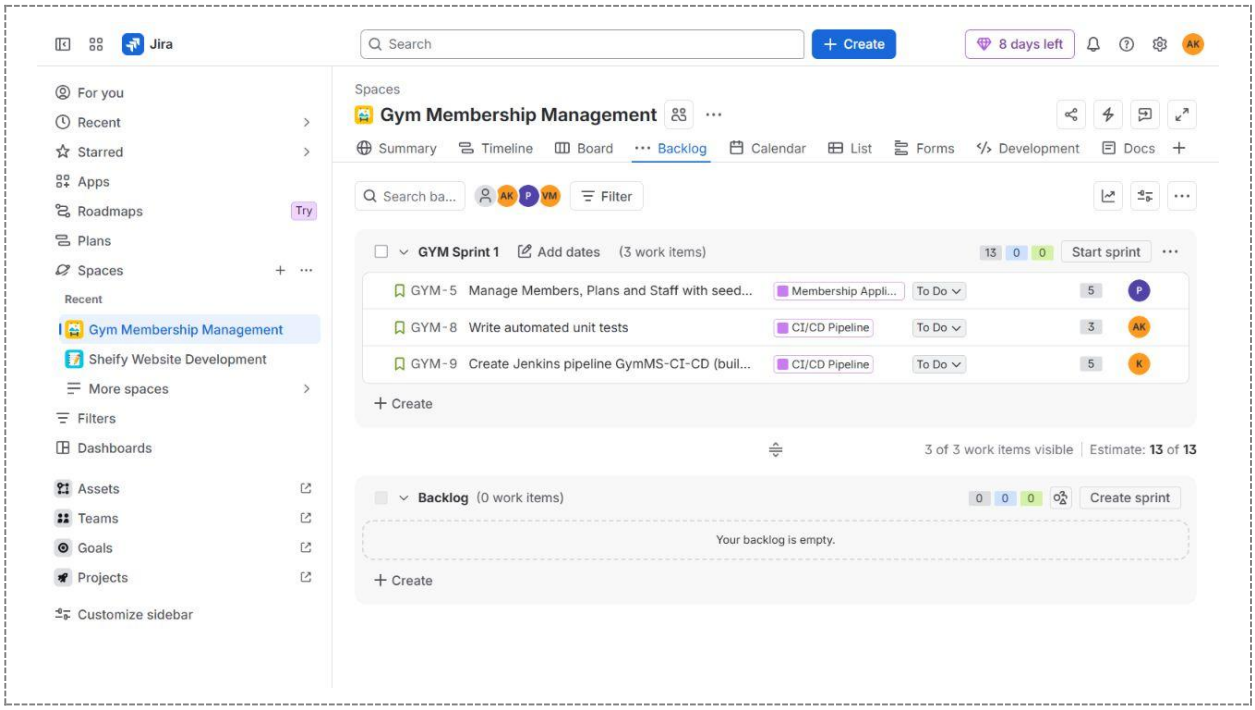


Figure 5.9: Live Jira backlog showing the three open sprint stories

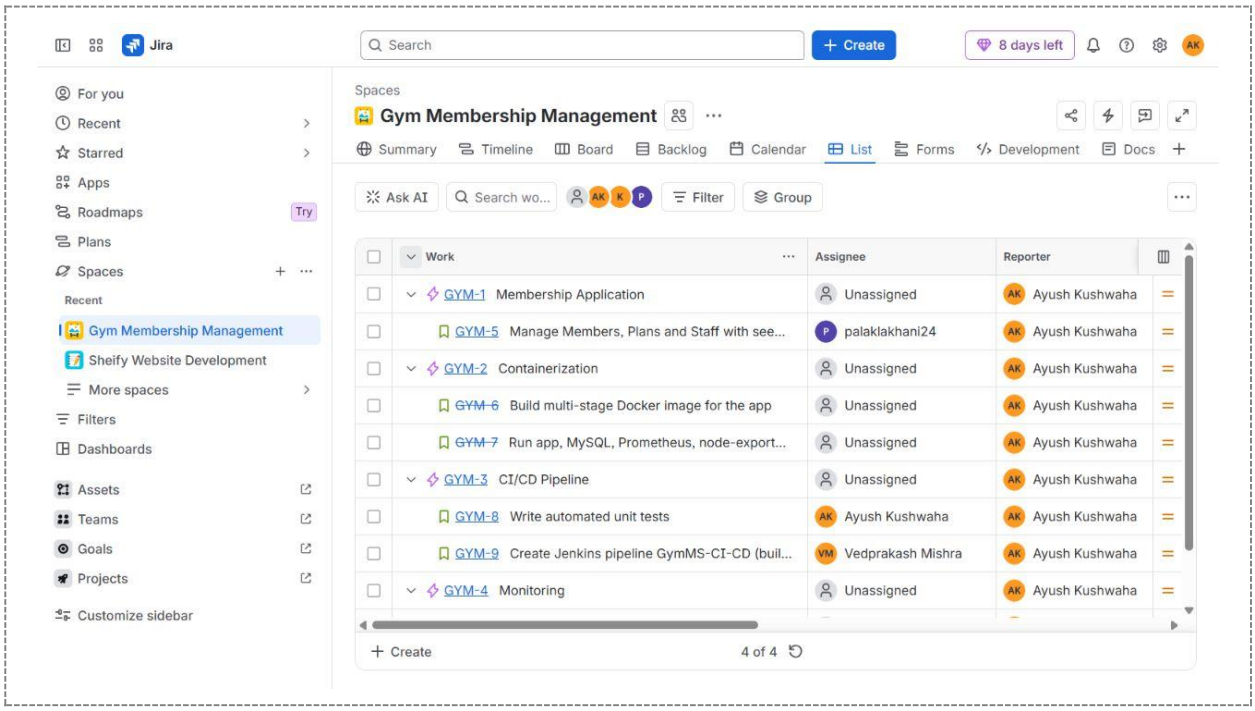


Figure 5.10: Jira project list with epics, stories and current statuses

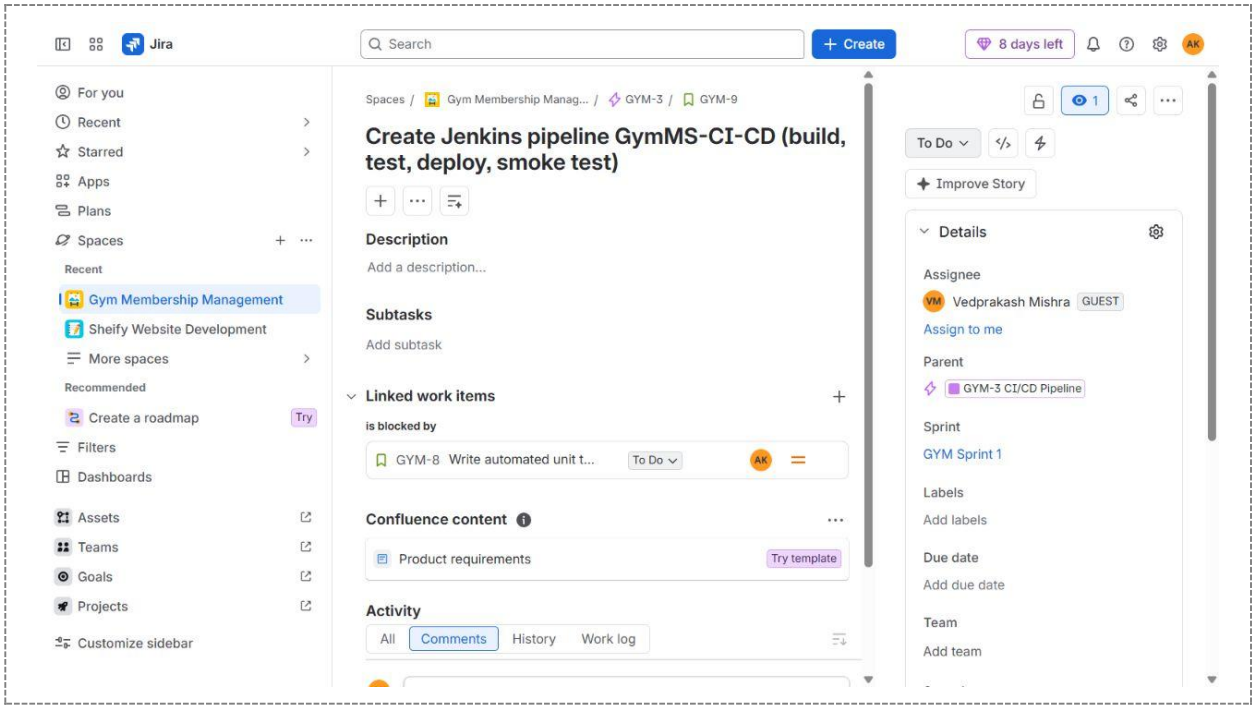


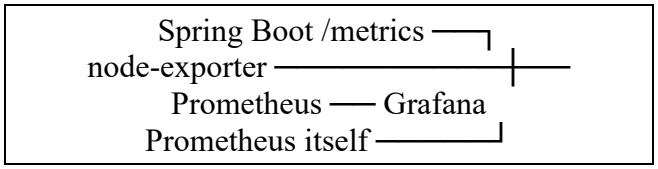
Figure 5.11: GYM-9 assigned to Vedprakash Mishra and blocked by GYM-8

5.7 Continuous Monitoring Using Prometheus — LO 5

Prometheus collects measurements over time, and Grafana displays them as counters and graphs. This project uses these two tools for monitoring; Nagios, MLflow and Airflow are not installed as part of the stack.

Spring Boot Actuator and Micrometer expose measurements at /metrics. Prometheus collects the app metrics, node-exporter metrics and its own metrics every 15 seconds. The four gym gauges count all members, Active members, plans and staff directly from the existing MySQL tables.

All three targets showed UP during verification. The provisioned GymMS Overview dashboard displays app scrape status, API request rate, p95 response time, Docker Linux VM CPU, memory and disk usage, and the four gym counters. The captured dashboard shows five members, four Active members, four plans and five staff records.



LO 5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios. The project implements the monitoring portion using Prometheus and Grafana.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  scrape_timeout: 10s

scrape_configs:
  - job_name: gymms
    metrics_path: /metrics
    static_configs:
      - targets: ['app:8080']
  - job_name: node-exporter
    static_configs:
      - targets: ['node-exporter:9100']
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']
```

Figure 5.12: Prometheus configuration from the project

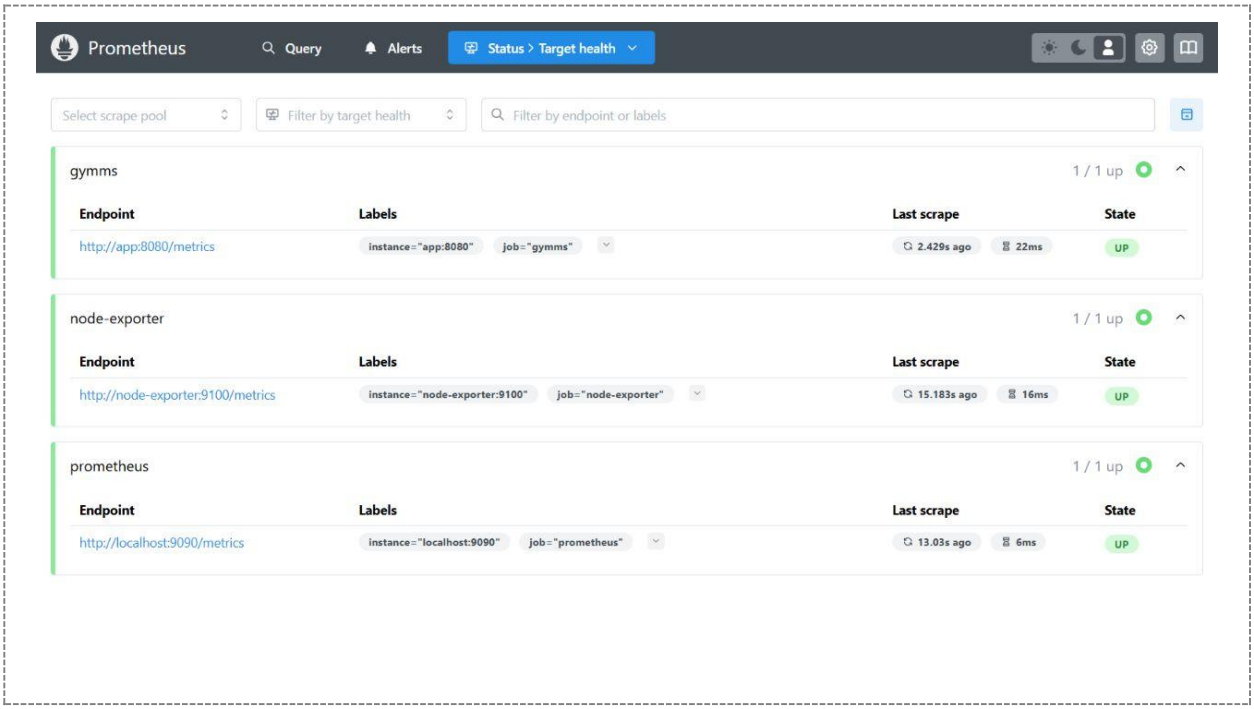


Figure 5.13: Prometheus targets showing all three services UP

```
up{job="gymms"}              1
up{job="node-exporter"}      1
up{job="prometheus"}         1
```

Figure 5.14: Target availability verified from Prometheus

```
gym_members_count{application="gymms"} 5.0
gym_members_active{application="gymms"} 4.0
gym_plans_count{application="gymms"} 4.0
gym_staff_count{application="gymms"} 5.0
```

Figure 5.15: Actual custom gauge values from /metrics

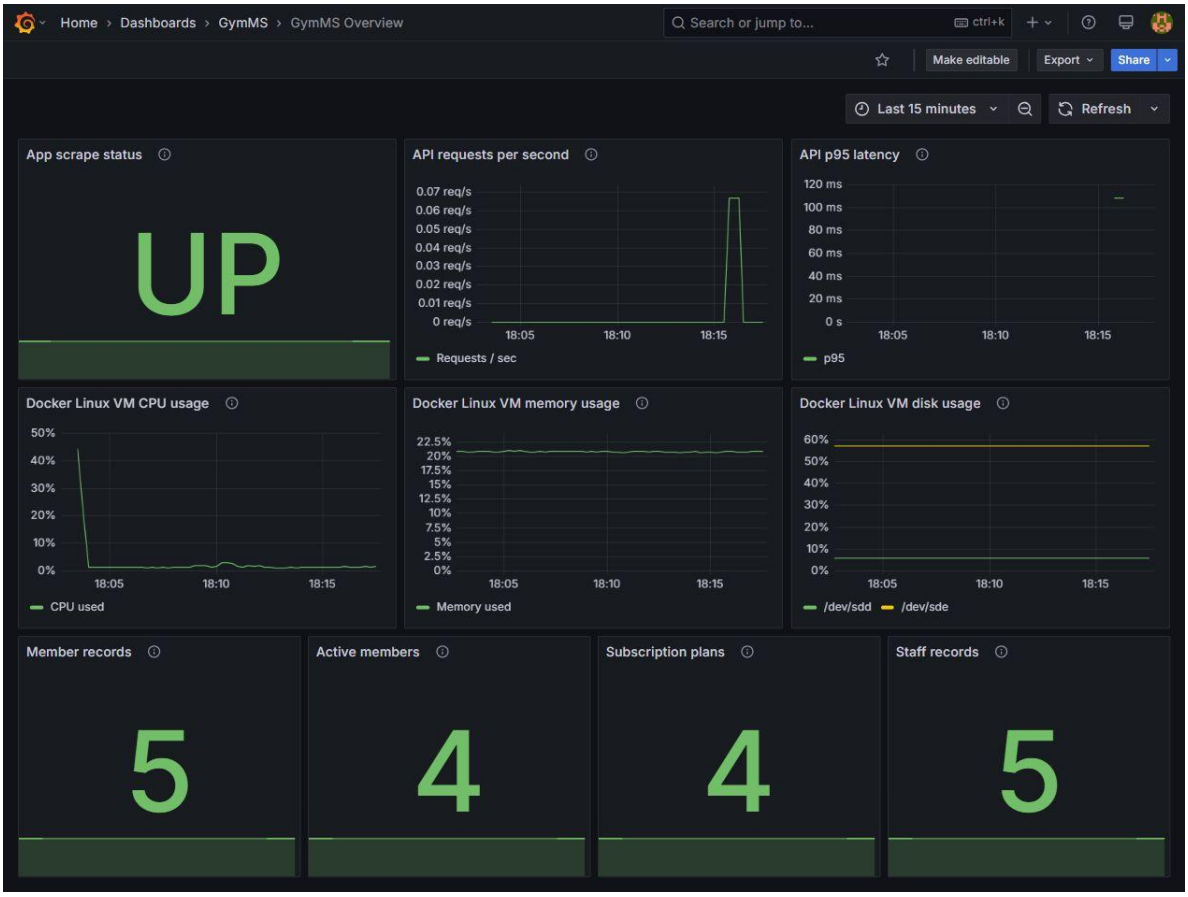


Figure 5.16: GymMS Overview dashboard with application and resource metrics

6. Testing

6.1 Automated Testing

The test suite contains six tests: context loading, plan CRUD, member CRUD with join-date preservation, staff CRUD, rejection of members with a missing plan, and rejection of deletion of an in-use plan. The test profile uses an in-memory H2 database with schema recreation and sample seeding disabled. Jenkins runs the tests in the Docker test stage and publishes the Surefire XML files as a JUnit report.

6.2 CI Verification

Build #1 published six passing tests, zero failures and zero skipped tests. Build #2 failed during its post-deployment health check, rather than during unit tests. The deliberately breakable test exists in source, but the planned failing-test/revert sequence is not recorded in the available build history.

6.3 Monitoring Verification

The Prometheus targets page and Grafana dashboard confirm that the monitoring services are receiving data. The raw app metrics include the four gym gauges. Requests made to the application are reflected in HTTP request counts; the dashboard's resource graphs refer to Docker's Linux environment.

6.4 Testing Evidence

The report includes live screenshots of GitHub, Jira, the application and monitoring. Git, Docker and Jenkins results are also reproduced as labelled command or API excerpts. Six automated tests passed in Jenkins build #1. Separate temporary API records were used to verify create, list, update and delete operations for all three entities and were removed afterwards.

```
GymManagementApplicationTests
  contextLoads
GymControllerIntegrationTests
  plansSupportCreateListUpdateAndDelete
  membersSupportCreateListUpdateAndDeleteWithoutChangingJoinDate
  pipelineRevertDemoRejectsMembersWithoutAnExistingPlan
  staffSupportCreateListUpdateAndDelete
  plansInUseCannotBeDeleted

Jenkins #1: 6 passed, 0 failed, 0 skipped
```

Figure 6.1: Automated tests recorded by Jenkins

7. Implementation:

The following screenshots show the application running at http://localhost:8080. The interface serves its own static frontend and uses relative /api/ URLs, so requests reach the same app that served the page.

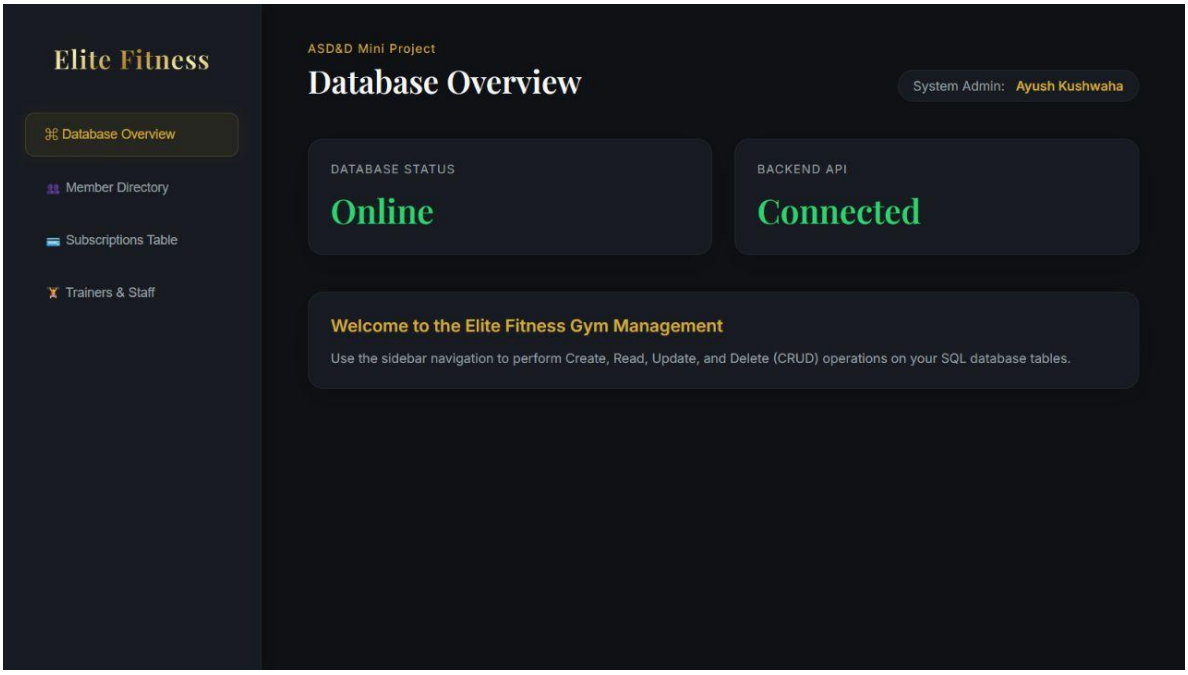


Figure 7.1: Gym Membership Management home page

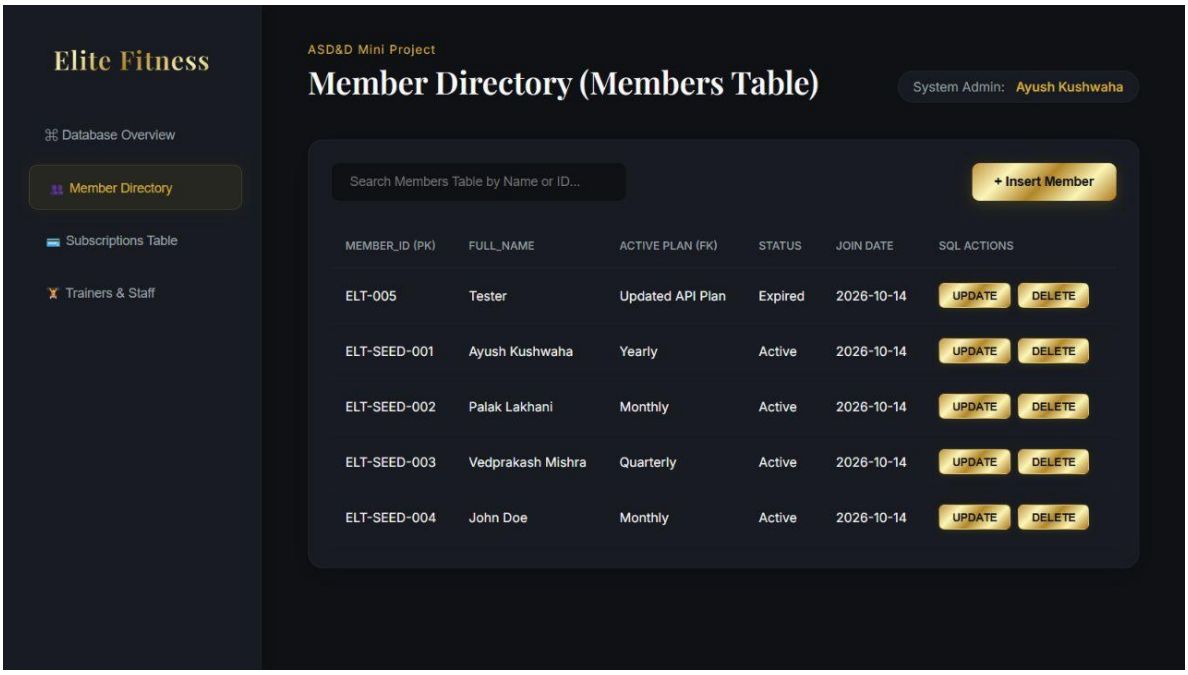


Figure 7.2: Member directory loaded from the application API

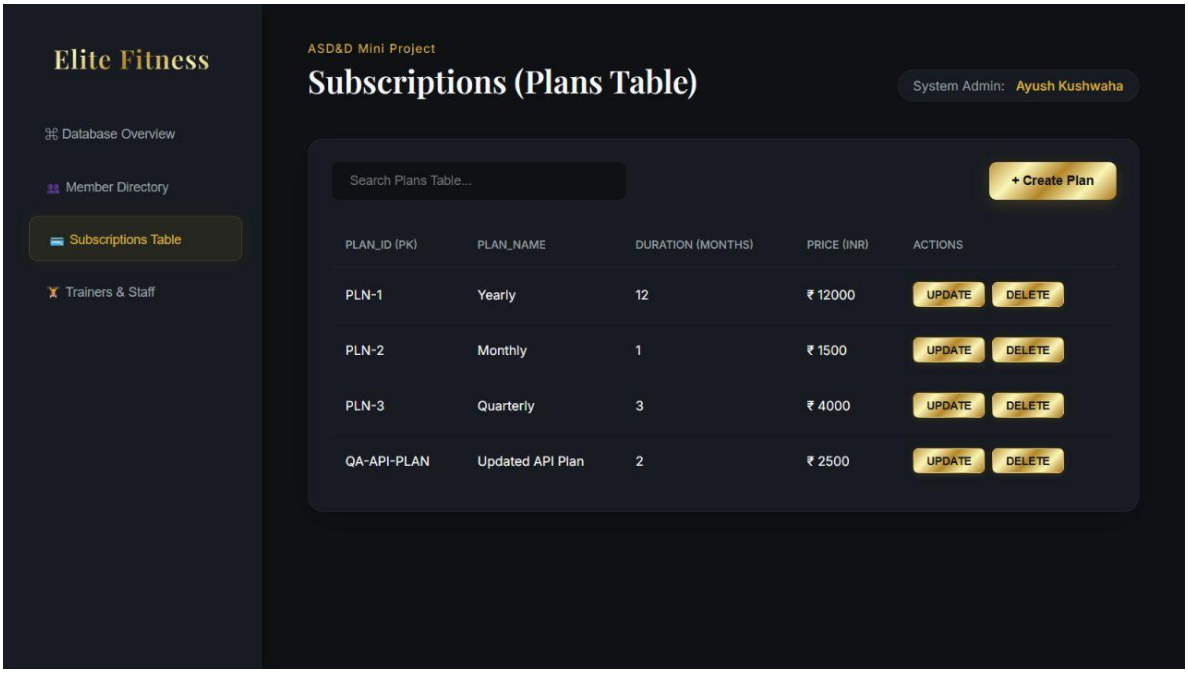


Figure 7.3: Subscription plans with duration and price

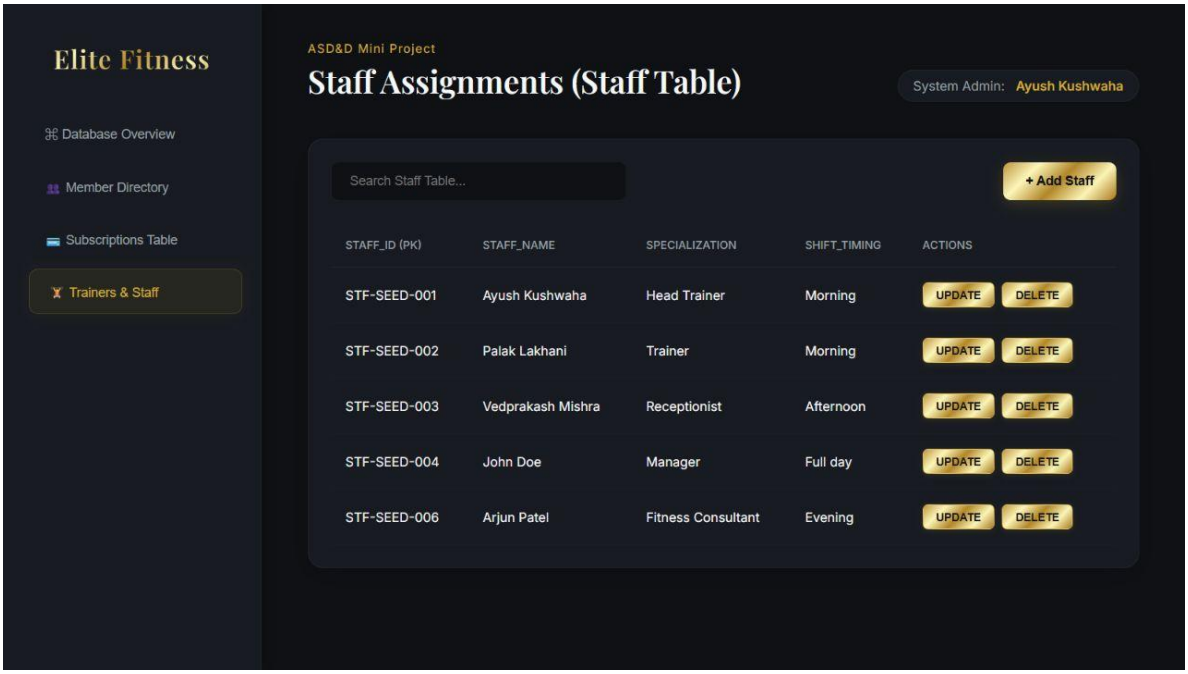


Figure 7.4: Staff records with specializations and shifts

8. Results and Discussion

8.1 Results

Area	Result
Version control	Two merged PRs: Docker and monitoring. Jenkins changes are on feature/jenkins; the local master merge is not yet on remote master.
Containerization	Java 21 multi-stage image; non-root production user; app container healthy.
Multi-service deployment	Five Compose services running with persistent database and monitoring volumes.
Continuous Integration	Jenkins build #1 passed six tests; polling watches feature/jenkins every two minutes.
Automated deployment	Build #1 succeeded. Build #2 failed at its post-deployment smoke test. The failed-test/revert demo is pending.
Agile workflow	Four epics, seven stories and two dependencies. Four stories are Done; three open stories appear in the unstarted sprint.
Continuous monitoring	Three Prometheus targets UP; Grafana displays app measurements and Docker Linux VM resource usage.

Table 8.1: Summary of project results

8.2 Discussion

The working application, database and monitoring services share one Compose environment. A source-code change is handled through Git and Jenkins; a member, plan or staff edit is handled by the application and MySQL. This separation allows normal record updates without rebuilding the application.

The implementation also shows why configuration and final verification matter. The local master and remote master are not yet synchronized, the Jenkins job still watches feature/jenkins, and the latest build failed during the smoke test. Jira's sprint has not started. These items must be resolved before presenting a fully completed delivery and sprint cycle.

The original member insert failure occurred because the form referenced plans that did not exist in Docker MySQL, and the frontend treated a failed HTTP response as a completed save. Dynamic plan choices, seeded data, backend validation and visible error messages address that failure. Updating a member now preserves the original join date.

9. Conclusion and Future Scope

9.1 Conclusion

The development of the Gym Membership Management project provided practical experience in applying software-development and DevOps concepts to a single integrated application.

Git and GitHub preserve source history, feature branches and two merged pull requests. Docker packages the Java application, and Compose runs it with MySQL and the monitoring services using persistent storage and health checks.

Jenkins executes tests and contains the build, deployment and health-check stages. A successful first build demonstrates the delivery path. The later smoke-test failure remains a concrete item for correction rather than evidence of a successful rollback demonstration.

Jira records four epics, seven stories and two dependency links. It provides named ownership for application verification, tests and the Jenkins pipeline. The sprint is prepared and requires dates and a start action before sprint progress can be demonstrated.

Prometheus and Grafana provide live visibility into the deployed application. Record counts, traffic, response times and resource usage can be observed without changing the main application features.

The project brings the application and its delivery tools together in a local environment. The remaining work is to align the release branch, correct the smoke-test wait and complete the planned sprint and rollback demonstration.

9.2 Future Scope

Immediate improvements are a reliable smoke-test retry, a release job watching master, a verified failed-test/revert sequence and completion of the Jira sprint. Later extensions could add authentication, backups, alerting, a staging environment or cloud deployment. Payment and attendance features would require separate requirements and implementation.

9.3 Learning Outcome — LO 6

LO 6: Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring. The project implements the connected local workflow; the results section identifies which demonstration steps still remain.

A. Appendix: Running the Project

A.1 Docker Build

```
docker build --target production -t gymms-app:local .
```

This builds the production image used by the app service.

A.2 Docker Compose

```
docker compose config --services
docker compose up -d --build
docker compose ps
docker compose logs
```

Run these commands from C:\gym-fitness-devops with Docker Desktop running. On a new installation, copy .env.example to .env and supply the required passwords. Do not overwrite a working .env or delete the existing database volume. The app uses port 8080, Prometheus 9090 and Grafana 3000. Grafana uses admin and the password in the local .env file.

A.3 Git

```
git log --oneline --graph
git remote -v
git push origin feature/jenkins
git revert <commit-to-undo>
```

The first commands inspect and publish code changes. The revert command is an example for a chosen faulty commit; it has not been used for the planned pipeline failure demonstration. Reverting code does not undo database deletions.

A.4 Monitoring

```
/metrics
up
rate(http_server_requests_seconds_count[1m])
```

Prometheus collects the application measurements from /metrics. The request counter can be queried with rate() to see requests per second. The configured scrape interval is 15 seconds.

A.5 Jenkins

Jenkins opens at http://localhost:8081/job/GymMS-CI-CD/. The job currently polls */feature/jenkins every two minutes. Start the existing installation with: java -jar "C:\Users\ayush\Downloads\jenkins.war" --enable-future-java --httpPort=8081 --httpListenAddress=127.0.0.1. Keep the terminal open for a foreground start.